1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Taller 2
Código de la guía (No.):	*002*
Taller(es) o Laboratorio(s) aplicable(s):	
Tiempo de trabajo práctico estimado:	6 horas
Asignatura(s) aplicable(s):	Programación Avanzada
Programa(s) Académico(s) / Facultad(es):	Ingeniería Electrónica / Ingeniería de Telecomunicaciones

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
C2: Capacidad para aplicar el diseño de ingeniería para generar soluciones que satisfagan necesidades específicas, considerando factores globales y económicos. C5: Capacidad para reconocer responsabilidades profesionales en situaciones de ingeniería y emitir juicios fundamentados al analizar e interpretar datos	Estructuras de datos avanzadas: Definición y manipulación de tuplas y diccionarios para la gestión de información estructurada. Modularización mediante funciones: Implementación de funciones con parámetros fijos, valores por defecto, y argumentos variables (*args y **kwargs). Programación Orientada a Objetos (POO): Creación de clases, definición de atributos y métodos, aplicación de herencia (super()) y sobrecarga de operadores.	Diseña procedimientos y soluciona problemas de ingeniería utilizando funciones, tuplas y diccionarios de manera eficiente. Diseña y evalúa clases que definen objetos para resolver problemas técnicos, integrando conceptos de herencia y sobrecarga de operadores. Aplica la lógica de programación para organizar grandes volúmenes de datos y modularizar soluciones algorítmicas complejas.

2. FUNDAMENTO TEÓRICO

El dominio de la programación en ingeniería trasciende el uso de estructuras de control básicas; requiere la capacidad de organizar la información de forma eficiente y modular. Las estructuras de datos como las tuplas y los diccionarios permiten al ingeniero representar sistemas complejos mediante agrupaciones de datos inmutables o relaciones de llave-valor, facilitando la gestión de inventarios, sensores o bases de datos académicas.

Por otro lado, la creación de funciones es el primer paso hacia la reutilización de código y la reducción de la redundancia, permitiendo que una solución sea escalable y fácil de mantener. Finalmente, la Programación Orientada a Objetos (POO) proporciona un paradigma que unifica datos (atributos) y comportamientos (métodos), permitiendo modelar entidades reales como cursos, estudiantes o presupuestos de investigación bajo criterios de herencia y abstracción profesional

3. OBJETIVO(S)

- Implementar estructuras de datos (tuplas y diccionarios) para el almacenamiento y procesamiento de información técnica en escenarios de ingeniería.

- Construir funciones modulares que empleen diversos tipos de parámetros para la resolución de problemas de cálculo y validación de datos.
- Aplicar los pilares de la Programación Orientada a Objetos (clases, herencia y sobrecarga) para modelar sistemas y componentes tecnológicos de forma estructurada.
- Utilizar entornos de desarrollo profesional como Google Colab o Visual Studio Code para traducir requerimientos de diseño en algoritmos funcionales y eficientes

4. RECURSOS REQUERIDOS

Los elementos requeridos para realizar el taller son:

- Computador personal con internet.
- Acceso a software tipo Visual Studio Code, Google Colab, Kagle Notebooks, entre otros.
- Material de estudio entregado en clase (Notebooks).

5. ASPECTOS DE SEGURIDAD

No aplica.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

6.1. Se tiene la tupla temperaturas = (22, 25, 28, 22, 24, 27, 22, 29, 30, 26, 22, 28, 23, 25) que representa las temperaturas máximas registradas durante las dos últimas semanas en una ciudad Escribe un programa que realice lo siguiente:

a) Muestre la temperatura más alta y la más baja del registro.

b) Calcule el promedio de temperatura de las dos semanas.

c) Cuente cuántas veces se registró la temperatura de 22 grados.

d) Intente modificar la primera temperatura a 25 grados y explique, mediante un comentario en el código, qué sucede al intentarlo.

6.2. Crea un diccionario llamado productos donde las llaves sean nombres de artículos de tecnología y los valores sean sus precios actuales. Inicializa el diccionario con: {"Laptop": 2500, "Mouse": 25, "Monitor": 300}. Realiza las siguientes operaciones de forma secuencial:

a) Agrega un nuevo producto: "Teclado" con un valor de 50.

b) El precio del "Monitor" ha subido; actualiza su valor a 350.

c) Elimina el producto "Mouse" del inventario.

d) Usa el método .get() para buscar el precio de una "Tablet". Si no existe, el programa debe mostrar un mensaje que diga "Producto no encontrado".

e) Imprime la lista final de productos y sus precios.

6.3. Se dispone de un diccionario de estudiantes donde la llave es el nombre del alumno y el valor es una **tupla** con tres notas estudiantes = {"Ana": (4.5, 3.8, 5.0), "Luis": (2.5, 3.0, 2.0), "Marta": (4.0, 4.2, 3.9), "Juan": (3.2, 2.8, 3.5)}. Escribe un script que:

a) Itere sobre el diccionario y calcule el promedio de notas de cada estudiante.

b) Muestre en pantalla el nombre del estudiante y su promedio final con dos decimales.

c) Al final, el programa debe imprimir cuántos estudiantes aprobaron (promedio ≥ 3.0) y cuántos reprobaron.

6.4. Eres el encargado de sistemas de una librería y tienes un inventario inicial representado por un diccionario donde la llave es el código del libro y el valor es otro diccionario con los detalles, tal que:

inventario = { "L001": {"titulo": "Python Pro", "precio": 150, "stock": 10}, "L002": {"titulo": "Data Science 101", "precio": 200, "stock": 5}, "L003": {"titulo": "Machine Learning", "precio": 300, "stock": 8} }

Escribe un script que:

a) Te llega un lote nuevo de libros, utiliza el método .update() para agregar un nuevo libro "L004" (título: "Deep Learning", precio: 350, stock: 12) y actualizar el precio del libro "L002" a 220.

b) Crea una lista de tuplas llamada ventas_dia que represente lo vendido hoy: [("L001", 3), ("L003", 2), ("L005", 1)]. Debes iterar sobre esta lista y descontar la cantidad vendida del stock en el diccionario inventario.

Reto: Como el código "L005" no existe, el programa debe usar .get() o una validación if para evitar que el código falle, imprimiendo un mensaje de advertencia: "Error: El producto L005 no existe en el inventario".

c) Crea un nuevo diccionario llamado valor_por_libro. Este debe tener como llave el título del libro y como valor el resultado de $precio \times stock$.

d) Imprime únicamente los títulos de los libros cuyo valor total calculado en el punto anterior sea superior a 2000.

6.5. Crea una función llamada convertir_distancia(valor, unidad="m").

a) La función debe recibir una distancia en metros y convertirla según el parámetro unidad.

b) Si unidad es "km", debe retornar la distancia en kilómetros.

c) Si unidad es "cm", debe retornar la distancia en centímetros.

d) Si no se especifica la unidad, el valor por defecto debe ser metros ("m") y retornar el valor original.

NOTA: Asegúrate de que el resultado retornado pueda ser usado en una operación matemática externa (ej. sumarle 10 al resultado).

6.6. Escribe una función llamada calcular_circulo(radio).

a) La función debe calcular tanto el área como el perímetro de un círculo.

b) Debe retornar ambos valores separados por coma.

c) Al llamar a la función, almacena los resultados en dos variables independientes (area y perimetro) usando el desempaquetado visto en clase.

d) Imprime un mensaje que diga: "El círculo con radio [X] tiene un área de [Y] y un perímetro de [Z]".

6.7. Diseña una función llamada promedio_notas(*calificaciones).

a) La función debe recibir cualquier cantidad de notas numéricas.

b) Internamente, debe tratar a calificaciones como una tupla.

c) Debe retornar el promedio simple de todas las notas ingresadas.

d) Si no se ingresa ninguna nota, la función debe retornar 0.0.

6.8. Como líder del semillero de investigación, necesitas una función que valide si tres mediciones de resistencia de un material son consistentes antes de guardarlas en la base de datos:

a) Define una variable fuera de la función llamada LIMITE_VARIACION = 20.5. Esta representará la diferencia máxima permitida entre la lectura más alta y la más baja.

b) Crea la función validar_ensayo(id_prueba, m1, m2, m3, **metadatos), donde:

- id_prueba: Un identificador (string).
- m1, m2, m3: Tres parámetros fijos (float) que representan las mediciones.
- **metadatos: Argumentos variables con nombre (ej. operador, laboratorio, fecha).

c) Verifica que m1, m2, m3 sean números. Si no, debe retornar un mensaje de error.

d) Calcula el promedio y la diferencia entre el valor máximo y el mínimo de las tres lecturas.

e) La función debe comparar esa diferencia con la variable global LIMITE_VARIACION.

f) La función debe entregar un diccionario que contenga:

- El id_prueba y el promedio calculado.
- Un valor booleano aprobado (True si la diferencia es menor o igual al límite global, False de lo contrario).
- Todos los datos contenidos en **metadatos integrados en el mismo diccionario

6.9. Crea una clase llamada Curso que permita gestionar la información básica de las asignaturas que dictas.

a) El constructor debe recibir los atributos de instancia nombre (str), el codigo (str) y el número de créditos (int).

b) Personaliza la salida para que al imprimir el objeto, se vea así: "Asignatura: [Nombre] | Código: [Código] | Créditos: [N]".

c) Crea un método llamado horas_estudio() que retorne el número de horas de estudio independiente recomendadas (calculadas como créditos × 3).

d) Crea dos objetos (instancia) correspondientes a dos cursos ("Programación Avanzada" y "Lógica de programación") e imprime sus datos y sus respectivas horas de estudio.

6.10. Especialización de Usuarios (Herencia y super())

a) Define una clase (padre) que reciba nombre y id_escolar en su constructor.

b) Define la clase (hija) Estudiante, que debe heredar de Persona. Utiliza super() para pasar el nombre y el ID de la clase padre. Además, inicializa de forma interna una lista vacía llamada self.notas.

c) Agrega el método 'agregar_nota(nota)'. Este método debe recibir un valor numérico y utilizar el método .append() para guardarlo en la lista self.notas.

d) La clase debe retornar una cadena con el nombre del estudiante y su promedio académico (usar __str__). Si el estudiante no tiene notas, el método debe indicar que el promedio es 0.0 para evitar errores de división por cero.

6.11. Suponga que, para tus proyectos de investigación, necesitas gestionar presupuestos y para ello va a implementar sobrecarga de operadores. Crea una clase llamada Presupuesto.

a) Recibe un valor numérico llamado monto.

b) Permite sumar dos objetos de tipo Presupuesto. La operación debe retornar una nueva instancia de Presupuesto con la suma de ambos montos.

c) Implementa el método necesario para poder usar el operador > (mayor que) entre dos presupuestos, comparando sus montos.

d) Prueba: Crea el presupuesto_investigacion (5.000.000) y el presupuesto_equipos (3.500.000). Súmalos para obtener un presupuesto_total y verifica mediante una sentencia if cuál de los dos presupuestos originales es mayor.

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

No aplica

8. DISPOSICIÓN DE RESIDUOS

No aplica

9. BIBLIOGRAFÍA

[1] https://github.com/estebangonzalezITM/ProgramacionAvanzada_2026-1
[2] https://python-reference. readthedocs.io/en/latest/docs/str/isalpha.html

Elaborado por:	*Esteban Gonzalez Valencia*
Revisado por:	*Comité de Inteligencia Computacional*
Versión:	*1*
Fecha:	*Marzo de 2026*